یک هفته قبل از آزمون برای همه بفرستید

🧰 چک‌لیست آماده‌سازی — چالش «بساز و منتشر کن»

این فایل را **یک هفته قبل از آزمون** بخوان و همه‌ی موارد را تا **یک روز قبل** روی لپ‌تاپ خودت آماده کن.
مشکل نصب در روز آزمون دلیل وقت اضافه نیست.

📅 درباره‌ی آزمون

- **زمان:** ۳ ساعت (به‌علاوه‌ی ۱۰ دقیقه وقت مطالعه قبل از شروع)، انفرادی، از یک پوشه‌ی خالی
- **موضوع:** یک میکروسرویس کوچک با Go می‌سازی و کامل منتشرش می‌کنی: تست، Bash، Docker، Docker Compose، GitHub Actions و Kubernetes
- **ابزار:** همه‌چیز آزاد است، حتی هوش مصنوعی؛ ولی بعد از آزمون یک **دفاع حضوری بدون AI** داری و باید هر خط کدت را توضیح بدهی
- **نمره:** ۸۰ ساخت + ۲۰ دفاع + ۱۵ امتیاز اضافه · **قبولی: ۶۰**
- **تحویل:** یک ریپازیتوری GitHub (عمومی، یا خصوصی با دعوت از حساب GitHub مدرس)

✅ ابزارها

ابزار	حداقل نسخه	دستور بررسی
Go	۱.۲۶ (جدیدترین نسخه بهتر است)	`go version`
کامپایلر C (فقط Linux و WSL، برای `go test -race`)	–	`gcc --version`
Git	هر نسخه‌ی جدید	`git --version`
حساب GitHub با کلید SSH	–	`ssh -T git@github.com`
Docker (نسخه‌ی Desktop یا Engine)	۲۴ به بعد	`docker run --rm hello-world`
Docker Compose v2	–	`docker compose version`
kubectl	۱.۲۸ به بعد	`kubectl version --client`
kind (پیشنهادی) یا minikube	–	`kind version` یا `minikube version`
curl	–	`curl --version`
Python 3	۳.۹	`python3 --version`
jq، redis-cli، psql، Helm	اختیاری	–

- در Ubuntu و WSL، کامپایلر C را این‌طور نصب کن (در macOS لازم نیست):

```
sudo apt update && sudo apt install -y build-essential
```

- **کاربران macOS:** ترمینال پیش‌فرض (zsh) علامت `#` را در خط فرمان کامنت حساب نمی‌کند و دستوری که در انتهایش `# …` باشد خطا می‌دهد. یک بار این را بزن و ترمینال را دوباره باز کن:

```
echo 'setopt interactivecomments' >> ~/.zshrc
```

☸️ کلاستر Kubernetes را از قبل بساز (و پاکش نکن)

```
kind create cluster --name exam
kubectl cluster-info --context kind-exam
kubectl get nodes
```

خروجی آخر باید نود `exam-control-plane` را با وضعیت `Ready` نشان بدهد. اگر minikube داری: `minikube start` و بعد `kubectl get nodes` .

📦 ایمیج‌ها را از قبل دانلود کن

تگ `golang` را با نسخه‌ی Go خودت هماهنگ کن: اگر `go version` می‌گوید `go1.27`، به‌جای خط اول بزن `docker pull golang:1.27-alpine`.

```
docker pull golang:1.26-alpine
docker pull alpine:3.22
docker pull postgres:16-alpine
docker pull redis:7-alpine
```

🐹 ماژول‌های Go را از قبل در cache بگذار

اگر اینترنتت محدود است، **اول** بخش «اگر اینترنت محدود است» پایین همین فایل را انجام بده. این دستورها را داخل پوشه‌ی یک پروژه اجرا نکن (هر بار که بزنی، بی‌خطر دوباره اجرا می‌شوند):

```
mkdir -p /tmp/warmup && cd /tmp/warmup && { [ -f go.mod ] || go mod init warmup; }
go get github.com/jackc/pgx/v5 github.com/redis/go-redis/v9
github.com/prometheus/client_golang/prometheus
```

> اگر این دستور خطای نسخه‌ی Go داد (مثلاً `requires go >= 1.26`)، Go را به‌روز کن. این دقیقاً همان خطایی است که نباید روز آزمون ببینی.

🪟 کاربران ویندوز

۱. WSL2 و Ubuntu را نصب کن: `wsl --install -d Ubuntu`

۲. در Docker Desktop به بخش Settings → Resources → WSL integration برو و Ubuntu را روشن کن.

۳. Go، kind، kubectl و `build-essential` را **داخل Ubuntu** نصب کن و همه‌ی کارها را همان‌جا انجام بده.

۴. برای جلوگیری از مشکل پایان خط اسکریپت‌ها: `git config --global core.autocrlf input`

🌐 اگر اینترنت محدود است (ایران)

- **Docker Hub** ممکن است IP ایران را مسدود کند. یک `registry-mirrors` تنظیم کن (آدرس mirror را مدرس اعلام می‌کند) یا از VPN استفاده کن:
  - در Docker Desktop: Settings → Docker Engine، و در JSON آن `"registry-mirrors": ["https://<mirror>"]` را اضافه کن.
  - در Docker Engine لینوکس: همین را در فایل `/etc/docker/daemon.json` بنویس و بعد `sudo systemctl restart docker` بزن.
  - بررسی: `docker info` باید زیر `Registry Mirrors` آدرس mirror را نشان بدهد.
- **Go proxy** (قبل از گرم کردن cache ماژول‌ها):

```
go env -w GOPROXY=https://goproxy.io,direct
```

- این تنظیم فقط روی Go سیستم خودت اثر دارد، نه داخل `docker build`. اگر روز آزمون دانلود ماژول‌ها داخل `docker build` شکست خورد، در مرحله‌ی ساخت Dockerfile و قبل از `RUN go mod download` این خط را بگذار: `ENV GOPROXY=https://goproxy.io,direct`
- ایمیج نود kind (`kindest/node`) هم از Docker Hub می‌آید، پس **کلاستر را حتماً از قبل بساز.**
- کلاستر kind از mirror تنظیم‌شده در Docker استفاده نمی‌کند. بعد از ساختن کلاستر و دانلود ایمیج‌ها، آن‌ها را از Docker خودت داخل کلاستر بارگذاری کن:

```
kind load docker-image postgres:16-alpine redis:7-alpine --name exam
```

- از ایمیج‌های `gcr.io` و `registry.k8s.io` دوری کن، چون اغلب مسدودند.

🧪 آزمایش نهایی (یک روز قبل)

این دستورها را اجرا کن و اگر مدرس خواست، خروجی را برایش بفرست:

```
go version
go env CGO_ENABLED
docker run --rm hello-world
kind get clusters
kubectl get nodes
python3 --version
ssh -T git@github.com
```

`go env CGO_ENABLED` باید `1` باشد. بعد `go test -race` را هم امتحان کن؛ باید `ok` چاپ شود:

```
cd /tmp/warmup && printf 'package warmup\n\nimport "testing"\n\nfunc TestOK(t *testing.T) {}\n' >
ok_test.go && go test -race .
```

اگر همه سبز بود، آماده‌ای. 🚀